

INFORME DE ENTREGABLE ETAPA 2: APLICACIÓN DE ESCRITORIO CON BASE DE DATOS

1. Nombre del Proyecto, Integrantes y Roles

- **Nombre del Proyecto:** *Car Racing - Sistema de Gestión de Carrera y Telemetría*
- **Integrantes del Grupo y Roles Asignados:**
 - **Dylan Almaraz:** *Desarrollador Backend / Base de Datos* — Configuración de SQLite, diseño de la estructura de la base de datos y lógica de operaciones CRUD.
 -
 - **Nicolás Herrera:** *Desarrollador Frontend / Interfaz Gráfica* — Maquetación con Tkinter, distribución de paneles y estilizado visual.
 -
 - **Thiago Gómez:** *QA & Documentación* — Control de calidad con la extensión SQLite Viewer en VS Code, prueba de casos borde y elaboración de la bitácora de problemas.
 -

2. Fundamentación de la Elección en el Diseño de la Aplicación

El diseño de la aplicación se estructuró para ofrecer una experiencia limpia, accesible y totalmente coherente con el proyecto desarrollado en la Etapa 1 (*Vehículo Robotizado Infrarrojo*):

- **Esquema Dark Mode (Racing Style):** Se eligió un fondo oscuro mate (`#121212` / `#1F1F1F`) combinado con acentos en rojo carrera (`#E50914`), verde victoria y azul. Este contraste emula las pantallas de telemetría profesional empleadas en boxes de automovilismo deportivo y reduce la fatiga visual.
-
- **Componente Interactivo Tabla (`ttk.Treeview`):** En sustitución del texto plano, se incorporó una tabla relacional estructurada con columnas y barra de desplazamiento vertical. Permite al usuario seleccionar cualquier fila cliqueando directamente sobre ella para cargar sus datos automáticamente en el formulario de edición.
-
- **Distribución Modular por Paneles:** Se organizó la pantalla en tres zonas bien delimitadas:
 - *Panel Izquierdo:* Formulario para carga y modificación de datos del piloto y del vehículo.
 -
 - *Panel Derecho:* Monitor/Tabla relacional para visualización de registros.
 -
 - *Panel Inferior:* Botonera de acciones CRUD con espaciado uniforme.
-

-
- **Menú Desplegable (Combobox):** Restringe las opciones del campo *Estado Vehículo* a valores estándar (*Óptimo, En Boxes, Ajustes de Motor, Fuera de Carrera*), evitando errores de tipeo manuales.

3. Código Creado con Comentarios

```

thon
import sqlite3
from tkinter import *
from tkinter import messagebox, ttk

# =====
# 1. BASE DE DATOS
# =====

conn = sqlite3.connect("./car_racing.db")
cursor = conn.cursor()

cursor.execute(
    """
    CREATE TABLE IF NOT EXISTS pilotos_carreras (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT NOT NULL,
        auto_modelo TEXT NOT NULL,
        mejor_tiempo REAL,
        vueltas_completadas INTEGER,
        estado_vehiculo TEXT
    )
    """
)
conn.commit()

# =====
# 2. VENTANA PRINCIPAL Y ESTILOS
# =====

Aplicacion = Tk()
Aplicacion.title("Car Racing Telemetry - Dashboard")
Aplicacion.geometry("1100x560")
Aplicacion.config(bg="#121212")

# Definición de la paleta de colores
BG_DARK = "#121212"
PANEL_BG = "#1F1F1F"
TEXT_COLOR = "#FFFFFF"
  
```

```

ACCENT_RED = "#E50914"
ACCENT_BLUE = "#00A8FF"
ACCENT_GREEN = "#00E676"
INPUT_BG = "#2C2C2C"

# Configuración de estilos para la tabla (Treeview)
style = ttk.Style()
style.theme_use("clam")
style.configure(
    "Treeview",
    background=INPUT_BG,
    foreground=TEXT_COLOR,
    fieldbackground=INPUT_BG,
    rowheight=25,
    font=("Segoe UI", 10),
)
style.configure(
    "Treeview.Heading",
    background=PANEL_BG,
    foreground=ACCENT_RED,
    font=("Segoe UI", 10, "bold"),
)
style.map("Treeview", background=[("selected", ACCENT_RED)])

# Header / Título Principal
title_frame = Frame(Aplicacion, bg=ACCENT_RED, height=60)
title_frame.pack(fill=X)

title_label = Label(
    title_frame,
    text="    CAR RACING TELEMETRY & MANAGEMENT",
    font=("Segoe UI", 18, "bold"),
    bg=ACCENT_RED,
    fg="white",
)
title_label.pack(pady=12)

# =====
# 3. PANEL DE FORMULARIO (IZQUIERDA)
# =====

Informacion = Frame(Aplicacion, bg=PANEL_BG, bd=2, relief=SOLID)
Informacion.place(x=15, y=75, width=420, height=360)

```

```

Label(
    Informacion,
    text="DATOS DEL PILOTO Y VEHÍCULO",
    font=("Segoe UI", 12, "bold"),
    bg=PANEL_BG,
    fg=ACCENT_RED,
).pack(pady=10)

form_container = Frame(Informacion, bg=PANEL_BG)
form_container.pack(fill=BOTH, expand=True, padx=15)

# Variables
num_id = StringVar()
nombre = StringVar()
auto_modelo = StringVar()
mejor_tiempo = StringVar()
vueltas = StringVar()
estado_vehiculo = StringVar()

def crear_campo(parent, label_text, var, row):
    Label(
        parent,
        text=label_text,
        font=("Segoe UI", 10, "bold"),
        bg=PANEL_BG,
        fg=TEXT_COLOR,
    ).grid(row=row, column=0, sticky=W, pady=6)
    entry = Entry(
        parent,
        textvariable=var,
        bg=INPUT_BG,
        fg="white",
        insertbackground="white",
        relief=FLAT,
        font=("Segoe UI", 10),
        width=22,
    )
    entry.grid(row=row, column=1, sticky=E, pady=6, padx=5)

crear_campo(form_container, "ID Registro:", num_id, 0)
crear_campo(form_container, "Piloto:", nombre, 1)

```

```

crear_campo(form_container, "Modelo Auto:", auto_modelo, 2)
crear_campo(form_container, "Mejor Tiempo (s):", mejor_tiempo, 3)
crear_campo(form_container, "Vueltas:", vueltas, 4)

# Combo para Estado del Vehículo
Label(
    form_container,
    text="Estado Vehículo:",
    font=("Segoe UI", 10, "bold"),
    bg=PANEL_BG,
    fg=TEXT_COLOR,
).grid(row=5, column=0, sticky=W, pady=6)
combo_estado = ttk.Combobox(
    form_container,
    textvariable=estado_vehiculo,
    values=["Óptimo", "En Boxes", "Ajustes de Motor", "Fuera de
Carrera"],
    width=20,
    state="readonly",
)
combo_estado.grid(row=5, column=1, sticky=E, pady=6, padx=5)

# =====
# 4. PANEL DE VISUALIZACIÓN / TABLA (DERECHA)
# =====
Fr1 = Frame(Aplicacion, bg=PANEL_BG, bd=2, relief=SOLID)
Fr1.place(x=450, y=75, width=635, height=360)

Label(
    Fr1,
    text="MONITOR DE TIEMPOS Y REGISTROS",
    font=("Segoe UI", 12, "bold"),
    bg=PANEL_BG,
    fg=ACCENT_RED,
).pack(pady=10)

# Configuración de la Tabla (Treeview)
columns = ("id", "piloto", "auto", "tiempo", "vueltas", "estado")
tabla = ttk.Treeview(Fr1, columns=columns, show="headings", height=10)

tabla.heading("id", text="ID")
tabla.heading("piloto", text="Piloto")
tabla.heading("auto", text="Modelo")

```

```

tabla.heading("tiempo", text="Tiempo (s)")
tabla.heading("vueltas", text="Vueltas")
tabla.heading("estado", text="Estado")

tabla.column("id", width=35, anchor=CENTER)
tabla.column("piloto", width=120)
tabla.column("auto", width=110)
tabla.column("tiempo", width=80, anchor=CENTER)
tabla.column("vueltas", width=60, anchor=CENTER)
tabla.column("estado", width=110, anchor=CENTER)

scroll_y = ttk.Scrollbar(Fr1, orient=VERTICAL, command=tabla.yview)
tabla.configure(yscrollcommand=scroll_y.set)

scroll_y.pack(side=RIGHT, fill=Y)
tabla.pack(fill=BOTH, expand=True, padx=10, pady=5)

# =====
# 5. FUNCIONES CRUD
# =====

def actualizar_tabla(registros=None):
    """Refresca las filas visualizadas en la tabla."""
    for item in tabla.get_children():
        tabla.delete(item)

    if registros is None:
        cursor.execute("SELECT * FROM pilotos_carreras")
        registros = cursor.fetchall()

    for row in registros:
        tabla.insert("", END, values=row)

def agregar():
    if nombre.get() == "" or auto_modelo.get() == "":
        messagebox.showwarning(
            "Atención", "Por favor ingrese al menos el Piloto y el
Auto."
        )
    return

```

```

try:
    t_val = float(mejor_tiempo.get()) if mejor_tiempo.get() else
0.0
    v_val = int(vueltas.get()) if vueltas.get() else 0
except ValueError:
    messagebox.showerror(
        "Error", "Tiempo y Vueltas deben ser valores numéricos."
    )
    return

cursor.execute(
    """
        INSERT INTO pilotos_carreras (nombre, auto_modelo,
mejor_tiempo, vueltas_completadas, estado_vehiculo)
        VALUES (?, ?, ?, ?, ?)
    """,
    (nombre.get(), auto_modelo.get(), t_val, v_val,
estado_vehiculo.get()),
)
conn.commit()
actualizar_tabla()
limpiar_campos()

def buscar():
    p_id = num_id.get()
    p_nombre = nombre.get()

    if p_id:
        cursor.execute("SELECT * FROM pilotos_carreras WHERE id = ?",
(p_id,))
    elif p_nombre:
        cursor.execute(
            "SELECT * FROM pilotos_carreras WHERE nombre LIKE ?",
            (f"%{p_nombre}%",),
        )
    else:
        cursor.execute("SELECT * FROM pilotos_carreras")

registros = cursor.fetchall()
actualizar_tabla(registros)

```

```

def eliminar():
    p_id = num_id.get()
    if not p_id:
        messagebox.showerror(
            "Error", "Seleccione o ingrese un ID para eliminar."
        )
        return

    cursor.execute("DELETE FROM pilotos_carreras WHERE id = ?",
(p_id,))
    conn.commit()
    actualizar_tabla()
    limpiar_campos()

def actualizar():
    p_id = num_id.get()
    if not p_id:
        messagebox.showerror(
            "Error", "Seleccione un registro existente para
actualizar."
        )
        return

    cursor.execute(
        """
        UPDATE pilotos_carreras
        SET nombre=?, auto_modelo=?, mejor_tiempo=?,
vueltas_completadas=?, estado_vehiculo=?
        WHERE id=?
        """,
        (
            nombre.get(),
            auto_modelo.get(),
            mejor_tiempo.get(),
            vueltas.get(),
            estado_vehiculo.get(),
            p_id,
        ),
    )
    conn.commit()
    actualizar_tabla()
    limpiar_campos()

```

```

def limpiar_campos():
    num_id.set("")
    nombre.set("")
    auto_modelo.set("")
    mejor_tiempo.set("")
    vueltas.set("")
    estado_vehiculo.set("")
    actualizar_tabla()

def al_seleccionar_tabla(event):
    """Carga los datos de la fila seleccionada directo en el
    formulario."""
    item = tabla.focus()
    if item:
        val = tabla.item(item, "values")
        num_id.set(val[0])
        nombre.set(val[1])
        auto_modelo.set(val[2])
        mejor_tiempo.set(val[3])
        vueltas.set(val[4])
        estado_vehiculo.set(val[5])

tabla.bind("<<TreeviewSelect>>", al_seleccionar_tabla)

# =====
# 6. PANEL DE BOTONES (INFERIOR)
# =====

Botones = Frame(Aplicacion, bg=PANEL_BG, bd=2, relief=SOLID)
Botones.place(x=15, y=450, width=1070, height=80)

def crear_boton(parent, text, command, bg_color, col):
    btn = Button(
        parent,
        text=text,
        command=command,
        bg=bg_color,
        fg="white",
        activebackground=bg_color,

```

```

        activeforeground="white",
        font=("Segoe UI", 10, "bold"),
        relief=FLAT,
        width=12,
        height=2,
        cursor="hand2",
    )
    btn.grid(row=0, column=col, padx=18, pady=12)

crear_boton(Botones, "  Agregar", agregar, ACCENT_GREEN, 0)
crear_boton(Botones, "  Buscar", buscar, ACCENT_BLUE, 1)
crear_boton(Botones, "  Actualizar", actualizar, "#FFB300", 2)
crear_boton(Botones, "  Eliminar", eliminar, ACCENT_RED, 3)
crear_boton(Botones, "  Limpiar", limpiar_campos, "#616161", 4)
crear_boton(
    Botones,
    "  Salir",
    lambda: (conn.close(), Aplicacion.destroy()),
    "#212121",
    5,
)

# Carga inicial de datos en la tabla
actualizar_tabla()

# Iniciar la app
Aplicacion.mainloop()

```

4. Bitácora de Problemas Encontrados y Soluciones Propuestas

#	Problema Encontrado	Causa Raíz	Solución Aplicada
1	Excepción en la base de datos: Ocurrió un error <code>sqlite3.OperationalError: no such table</code> al abrir la app por primera vez.	El archivo <code>.db</code> se creaba en el disco pero no contenía la estructura de la tabla previa a la primera inserción.	Se incorporó la instrucción SQL <code>CREATE TABLE IF NOT EXISTS</code> durante la inicialización de la base de datos.

2	Error de tipo ValueError: La aplicación fallaba si se ingresaban letras o caracteres en las cajas de mejor tiempo o número de vueltas.	Intentar convertir entradas no numéricas directamente a datos <code>float</code> o <code>int</code> sin validar.	Se protegió el procesamiento mediante un bloque <code>try-except ValueError</code> que notifica al usuario con un <code>messagebox.showerror</code> .
3	Selección manual lenta: Para actualizar o eliminar registros, el usuario debía ingresar manualmente el ID número por número.	Ausencia de enlace dinámico entre la tabla visual de registros y las cajas de texto del formulario.	Se asoció el evento <code><<TreeviewSelect>></code> a la tabla, relleno de forma automática todos los campos al hacer clic en cualquier fila.
4	Inconsistencia de datos: Los datos insertados en la interfaz no se reflejaban de inmediato al consultar la tabla con SQLite Viewer en VS Code.	Omitir la instrucción de confirmación de la transacción en la base de datos de SQLite.	Se agregó la llamada explícita <code>conn.commit()</code> tras realizar cualquier operación <code>INSERT</code> , <code>UPDATE</code> o <code>DELETE</code> .